

Informática I

Unidad 2: Algoritmos Secuenciales y Condicionales

Cristian A. Jimenez C.

Departamento de Ingeniería Electrónica y Telecomunicaciones
Facultad de Ingenierías
Universidad de Antioquía

cjimenez.castano@udea.edu.co

Oficina 20-405

Contenido I

- 1 Solución de Problemas
 - Método de Polya
- 2 Algoritmos
 - Diagramas de flujo (introducción)
- 3 Algoritmos en Python
 - Algoritmos en Python
 - Operadores y primitivas
 - Variables y tipos de datos numéricos
 - Entrada y salida (input, print)
 - De diagrama a código
- 4 Ejecución de Programas
 - Principio de Ejecución Secuencial
 - Estado de un Programa
 - Ejecución Concurrente
- 5 Strings
 - Strings en Python
 - Indexación y Segmentación de Strings
- 6 Condicionales

Contenido II

- Condicionales y Variables Lógicas (if, else)
- Condicionales Anidados (elif, if-then-if)
- Operadores Lógicos
- Operador in

7 Prueba de Escritorio

8 Bibliografía

Contenido I

- 1 Solución de Problemas
 - Método de Polya
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales
- 7 Prueba de Escritorio
- 8 Bibliografía

Solución de Problemas

A partir del planteamiento de un problema, oportunidad, o dificultad, es conveniente usar una **metodología** para la resolución del mismo, que tendrá como objetivo final la **fabricación** del **algoritmo** que dará la **solución**.



George Pólya, matemático húngaro, autor del libro *How to solve it* [2]. En él, Pólya propone cuatro (4) pasos generales para la solución de un problema.



1. Entienda el Problema

Parece obvio pero es con frecuencia un gran obstáculo.

- 👉 ¿Entiende todas las palabras de la formulación del problema?
- 👉 ¿Qué le están pidiendo que encuentre?
- 👉 ¿Puede usted reformular el problema en sus propias palabras?
- 👉 ¿Hay suficiente información para encontrar la solución?

2. Diseña un Plan

Escoja una estrategia para resolver el problema.

divida el problema en problemas más simples, elimina posibilidades, aproveche simetrías, suponga y verifique, etc.

3. Implemente el plan

Más fácil que el paso 2, sólo requiere mucho cuidado en los detalles y paciencia.

4. Revise

Haga una pausa, revise y reflexione sobre el trabajo realizado.

Ejemplo 1: un problema de razón de cambio

Problema: Un tanque tiene capacidad para 200 litros y ya contiene 50 litros. Una manguera lo llena a razón de 5 litros por minuto. ¿Cuántos minutos faltan para que el tanque esté lleno?

Ejemplo 1: un problema de razón de cambio

Problema: Un tanque tiene capacidad para 200 litros y ya contiene 50 litros. Una manguera lo llena a razón de 5 litros por minuto. ¿Cuántos minutos faltan para que el tanque esté lleno?

- 1 **Entender:** datos = capacidad (200 L), cantidad actual (50 L), tasa de llenado (5 L/min). Pregunta = minutos que faltan.
- 2 **Plan:** primero calcular cuánta agua falta (capacidad – actual); luego dividir esa cantidad entre la tasa de llenado.
- 3 **Ejecutar:**

$$\text{Falta} = 200 - 50 = 150 \text{ L}$$

$$\text{Tiempo} = 150 \div 5 = \mathbf{30 \text{ minutos}}$$

- 4 **Revisar:** $50 + 5 \times 30 = 50 + 150 = 200 \text{ L}$. ✓ El resultado es positivo y razonable.

Ejemplo 2: un problema de lógica

Problema: Hoy es lunes. ¿Qué día de la semana será dentro de 100 días?

Ejemplo 2: un problema de lógica

Problema: Hoy es lunes. ¿Qué día de la semana será dentro de 100 días?

- 1 **Entender:** los días de la semana se repiten cada 7 días. Solo me importa *cuántos días “sobran”* después de completar semanas enteras.
- 2 **Plan:** dividir 100 entre 7; el **residuo** me dice cuántos días avanzar desde hoy.
- 3 **Ejecutar:**

$$\begin{aligned}100 \div 7 &= 14 \text{ semanas completas, residuo } 2 \\(14 \times 7 &= 98, 100 - 98 = 2) \\ \text{Lunes} + 2 \text{ días} &= \mathbf{Miércoles}\end{aligned}$$

- 4 **Revisar:** probemos con un caso pequeño: 7 días después de un lunes debe volver a ser lunes (residuo 0). ✓ El método funciona.

Esa operación de “residuo” es el operador % (módulo) que usarán en Python más adelante.

Ahora inténtalo tú: aplica las 4 fases de Pólya

Para cada problema, identifica explícitamente los 4 pasos: entender, planear, ejecutar y revisar.

- a) **(Matemático)** Compraste 3 cuadernos y 2 lápices por \$17 000. Un cuaderno cuesta el doble que un lápiz. ¿Cuánto cuesta cada uno?
- b) **(Lógico)** Un caracol debe subir un poste de 10 metros. Cada día sube 3 metros, pero cada noche resbala 2 metros. ¿Cuántos días tarda en llegar a la cima?
- c) **(Lógico)** Si hoy es miércoles, ¿qué día de la semana será dentro de 365 días?

Sugerencia: el problema (c) se resuelve con la misma idea del Ejemplo 2 — y el (b) tiene una trampa: revisa bien qué pasa el último día.

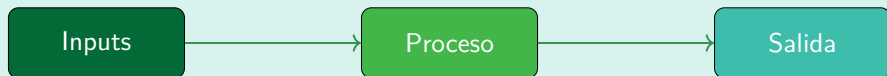
Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
 - Diagramas de flujo (introducción)
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales
- 7 Prueba de Escritorio
- 8 Bibliografía

Definición

Recuerde que un algoritmo es un conjunto de instrucciones ordenadas, el cual es preciso, computable y finita, que paso a paso llevan a la solución de un problema.

Todo algoritmo tiene:



¿Cómo ingresar a la Universidad de Antioquia?

- 1 Comprar formulario de inscripción.
- 2 Elegir carrera.
- 3 Presentar examen.
- 4 Si no pasa, volver al paso 1.
- 5 Pagar matrícula.
- 6 Elegir materias.

¿Cómo dibujar una parabola en el plano cartesiano $(-10,10)$?

- 1 Asignar a x el valor de -10 .
- 2 Asignar a y el valor de x^2 .
- 3 Dibujar un punto en la coordenada (x, y) .
- 4 Sumar 1 a x .
- 5 Si x es menor o igual a 10 , vaya al paso 2.

- **¿Cuál es el objetivo buscando?**
Calcular el área de un triángulo.

- **¿Cuál es el objetivo buscando?**
Calcular el área de un triángulo.
- **¿Cuáles son los datos de entrada?**
Base y altura.

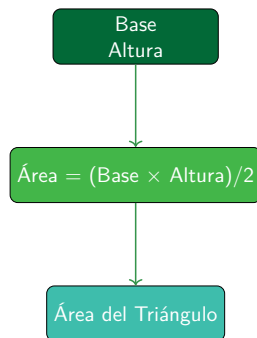
- **¿Cuál es el objetivo buscando?**
Calcular el área de un triángulo.
- **¿Cuáles son los datos de entrada?**
Base y altura.
- **¿Cuáles son los datos de salida?**
Área de un triángulo.

- **¿Cuál es el objetivo buscando?**
Calcular el área de un triángulo.
- **¿Cuáles son los datos de entrada?**
Base y altura.
- **¿Cuáles son los datos de salida?**
Área de un triángulo.
- **¿Qué cálculos/procesos deben de llevarse a cabo?**

$$A_{\Delta} = \frac{Base \times Altura}{2}$$

- **¿Cuál es el objetivo buscando?**
Calcular el área de un triángulo.
- **¿Cuáles son los datos de entrada?**
Base y altura.
- **¿Cuáles son los datos de salida?**
Área de un triángulo.
- **¿Qué cálculos/procesos deben de llevarse a cabo?**

$$A_{\Delta} = \frac{Base \times Altura}{2}$$



Data en un algoritmo

Variable

Espacio de memoria al que se le da un nombre y que almacena un valor (un dato) que puede ser modificado por instrucciones del algoritmo.

Data en un algoritmo

Variable

Espacio de memoria al que se le da un nombre y que almacena un valor (un dato) que puede ser modificado por instrucciones del algoritmo.

Tipos de variables

- Variables de entrada y salida.
- Variables auxiliares.
- **Constante:** un dato que no cambia.

Data en un algoritmo

Variable

Espacio de memoria al que se le da un nombre y que almacena un valor (un dato) que puede ser modificado por instrucciones del algoritmo.

Tipos de variables

- Variables de entrada y salida.
- Variables auxiliares.
- **Constante:** un dato que no cambia.

Una variable puede representar un número decimal, un número entero, un arreglo de números o de caracteres, etc.

Ejemplo de Algoritmo I

Se requiere diseñar un algoritmo que **calcule el número de meses que hay entre los años A y B.**

Datos de entrada: los años especificados (A y B).

Datos de salida: número total de meses transcurridos.

Definición de variables:

- A: primer año
- B: segundo año
- years: años transcurridos
- months: meses transcurridos

Ejemplo de Algoritmo II

Algoritmo

- 1 Capturar valores de A y B
- 2 Asignar a $years$ la operación $B - A$
- 3 Asignar a $months$ la operación $years \times 12$
- 4 Mostrar el valor de $months$

Diagramas de Flujo

Definición

Representación **gráfica** de un **algoritmo** o proceso. También conocido como **flujograma** o **diagrama de actividades**.

¿Para qué nos sirve?

Es una herramienta de **planeación**: antes de escribir código, dibujamos cómo fluye la solución. **Más adelante, cuando veamos condicionales**, la usaremos con más detalle. Por ahora conozcamos sus símbolos básicos.

Diagramas de Flujo: símbolos básicos

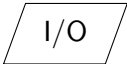
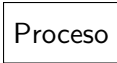
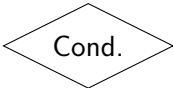
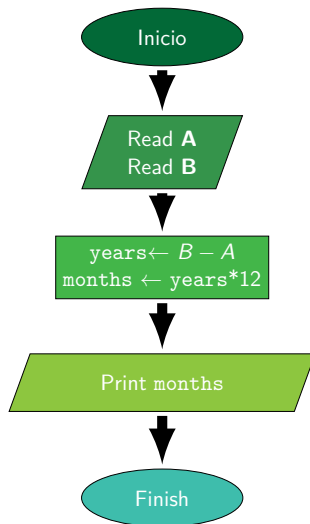
Símbolo	Descripción
	Representa el punto de inicio o final del proceso.
	Representa operaciones de entrada/salida (leer o imprimir datos).
	Representa un proceso o cálculo que se realiza en el diagrama.
	Representa una decisión que bifurca el flujo (condicionales) o que controla la repetición de pasos (ciclos).
	Indican la secuencia entre dos pasos del algoritmo.

Diagrama de Flujo: meses entre A y B

Recordemos el ejemplo

Algoritmo que calcula el número de meses que hay entre los años A y B .

- 1 Capturar valores de A y B .
- 2 Asignar a $years$ la operación $B - A$.
- 3 Asignar a $months$ la operación $years * 12$.
- 4 Mostrar el valor de $months$.



El pseudocódigo y el diagrama dicen lo mismo, de dos formas distintas. Muy pronto los llevaremos a código real de Python.

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
 - Algoritmos en Python
 - Operadores y primitivas
 - Variables y tipos de datos numéricos
 - Entrada y salida (`input`, `print`)
 - De diagrama a código
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales

Contenido II

7 Prueba de Escritorio

8 Bibliografía

¿Qué es Python?

Definición

Python es un lenguaje de programación de **alto nivel**, **interpretado** y de propósito general, con una sintaxis clara y cercana al lenguaje natural.

- Vamos a usarlo para **ejecutar** los algoritmos que hasta ahora solo planeábamos en pseudocódigo o diagramas de flujo.
- Se puede ejecutar de dos formas: escribiendo un archivo `.py` (script) o de forma interactiva en un intérprete.
- Los comentarios (texto que Python ignora, para explicar el código) inician con `#`.

Nuestro primer programa

```
# Mi primer programa en Python  
print("Hola, mundo")
```


Sintaxis Versus Semántica (en Python)

Sintaxis

Conjunto de reglas que determinan qué combinaciones de símbolos son válidas en el lenguaje.

```
# Correcto  
print(x + 2)
```

```
# Error de sintaxis  
print x + 2
```

Semántica

Es el **significado** de las instrucciones: qué se quiere lograr, más allá de cómo se escriba el código.

Validar si un número es par.

Sintaxis Versus Semántica (en Python)

Sintaxis

Conjunto de reglas que determinan qué combinaciones de símbolos son válidas en el lenguaje.

```
# Correcto  
print(x + 2)
```

```
# Error de sintaxis  
print x + 2
```

Semántica

Es el **significado** de las instrucciones: qué se quiere lograr, más allá de cómo se escriba el código.

Validar si un número es par.

Suele requerir pensar en la lógica del problema aparte (por ejemplo, usando el operador %) antes de traducirla a instrucciones de Python.

Operadores y Primitivas en Python

- Son operaciones que Python ya sabe ejecutar: **no** es necesario programarlas nosotros mismos.
- El **intérprete** de Python las reconoce directamente al leer el código.

Operadores aritméticos			
Operador	Nombre	Ejemplo	Resultado
+	Suma	1+3	4
-	Resta	7-10	-3
*	Multiplicación	3*5	15
/	División	8/5	1.6
//	División entera	8//5	1
%	Modulo (residuo)	15%7	1
**	Potencia	2**3	8

Operadores y Primitivas en Python

- Son operaciones que Python ya sabe ejecutar: **no** es necesario programarlas nosotros mismos.
- El **intérprete** de Python las reconoce directamente al leer el código.

Operadores relacionales			
Operador	Nombre	Ejemplo	Resultado
>	Mayor	1>3	False
>=	Mayor o igual	2>=1	True
<	Menor	3<3	False
<=	Menor o igual	3<=3	True
!=	Diferente	13!=4	True
==	Igual	0==1	False

Operadores y Primitivas en Python

- Son operaciones que Python ya sabe ejecutar: **no** es necesario programarlas nosotros mismos.
- El **intérprete** de Python las reconoce directamente al leer el código.

Así se ven en Python:

```
a = 15
b = 7

print(a + b)      # 22
print(a % b)      # 1
print(a ** 2)     # 225

print(a > b)       # True
print(a == b)     # False
```

Jerarquía de Operaciones

Orden de Evaluación

Las operaciones se evalúan siguiendo esta jerarquía:

- ➊ **Paréntesis:** Se evalúan primero.
- ➋ **Exponentes:** Potenciación y raíces.
- ➌ **Multiplicación y División:** Se evalúan de izquierda a derecha.
- ➍ **Suma y Resta:** Se realizan al final.
- ➎ **Operadores Relacionales y Lógicos:** Se evalúan después de las operaciones aritméticas.

Jerarquía de Operaciones

Orden de Evaluación

Las operaciones se evalúan siguiendo esta jerarquía:

- ➊ **Paréntesis:** Se evalúan primero.
- ➋ **Exponentes:** Potenciación y raíces.
- ➌ **Multiplicación y División:** Se evalúan de izquierda a derecha.
- ➍ **Suma y Resta:** Se realizan al final.
- ➎ **Operadores Relacionales y Lógicos:** Se evalúan después de las operaciones aritméticas.

Ejemplo

Realizar la operación:

$$\underbrace{3 + 4 \times 2^2 - \frac{6 - 2}{2 - 5}}_{\text{Forma Matemática}} \Leftrightarrow \underbrace{3 + 4 * (2 ** 2) - (6 - 2) / (2 - 5)}_{\text{En Python}}$$

¡Ahora inténtalo tú!

Ejercicio

Traduce la siguiente expresión matemática a una **expresión válida en Python**, usando los operadores primitivos y respetando la jerarquía de operaciones:

$$\frac{5^2 - 3}{2} + \frac{(4 + 1) \times 3}{7 - 2}$$

Pistas

- En la expresión matemática las fracciones “agrupan” el numerador y el denominador; en Python esa agrupación hay que hacerla **explícita** con paréntesis.
- ¿Qué operador de Python usarías para escribir 5^2 ?
- Verifica el resultado paso a paso, tal como en el ejemplo anterior.

Operador de Asignación en Python

- Una expresión por sí sola no indica qué hacer con el resultado

$$((x * 3) - 5 * x - 2) / (a \% d) * k$$

- En los diagramas usamos la flecha $\leftarrow$ para *asignar*. En **Python el operador de asignación es el signo igual =**.

```
z = ((x**3) - 5*x - 2) / (a % d) * k
```

¡Importante! = no es ==

= **asigna** un valor a una variable (cambia su estado). == **compara** dos valores y devuelve True o False. Confundirlos es uno de los errores más comunes al empezar a programar.

Variables en Python

Recordemos

Una **variable** es un espacio de memoria, con nombre, que almacena un valor que puede cambiar.

```
edad = 20
```

En Python no se declara el tipo

No escribimos `int edad = 20` como en otros lenguajes: Python **infiere el tipo** automáticamente a partir del valor asignado (tipado dinámico).

Reglas para nombrar variables

- Solo letras, números y guion bajo `_`; no puede iniciar con un número.
- Es sensible a mayúsculas: `edad` $\neq$ `Edad`.
- No se puede usar una palabra reservada (`if`, `for`, `print`, ...).
- Convención recomendada: `snake_case` (ej. `año_nacimiento`).

Tipos de Datos en Python

int

Números enteros, positivos o negativos, sin parte decimal.

```
edad = 20
```

float

Números con parte decimal (punto flotante).

```
altura = 1.75
```

str

Texto (cadena de caracteres), entre comillas.

```
nombre = "Ana"
```

type(): ¿de qué tipo es una variable?

```
type(edad)           # <class 'int'>
type(altura)         # <class 'float'>
type(nombre)         # <class 'str'>
type(edad + altura)  # <class 'float'>
```

Conversión de tipos

```
int(3.9) → 3 (trunca)   float(5) → 5.0   str(20) → "20"
```

La función print()

¿Para qué sirve?

Es la instrucción de **salida**: muestra información en pantalla.

```
print("Hola")           # Hola
print("x =", 5)          # x = 5
print(1, 2, 3, sep="-")  # 1-2-3
print("Sin salto", end=" ") # controla que va
    al final
print("de linea.")
```

Nota

print() puede recibir varios valores separados por comas; por defecto los separa con un espacio.

La función `input()`

¿Para qué sirve?

Es la instrucción de **entrada**: pausa el programa y espera que el usuario escriba algo por teclado.

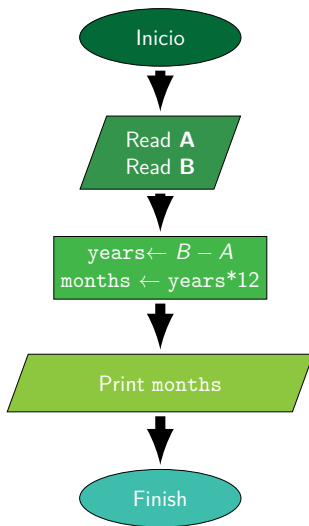
```
nombre = input("Como te llamas? ")
```

¡Importante!

`input()` **siempre** devuelve un `str` (texto), aunque el usuario escriba un número. Para operar ese valor como número hay que convertirlo con `int()` o `float()`.

```
edad = int(input("Cuantos años tienes? "))  
print(nombre, "el proximo año tendras", edad + 1)
```

Ejemplo: meses entre A y B, del diagrama al código I



Ejemplo: meses entre A y B, del diagrama al código II

Traduciendo el diagrama a Python

Cada bloque del diagrama se convierte en una o más líneas de código:

```
# Meses transcurridos entre dos años
A = int(input("Año inicial (A): "))
B = int(input("Año final (B): "))

years = B - A
months = years * 12

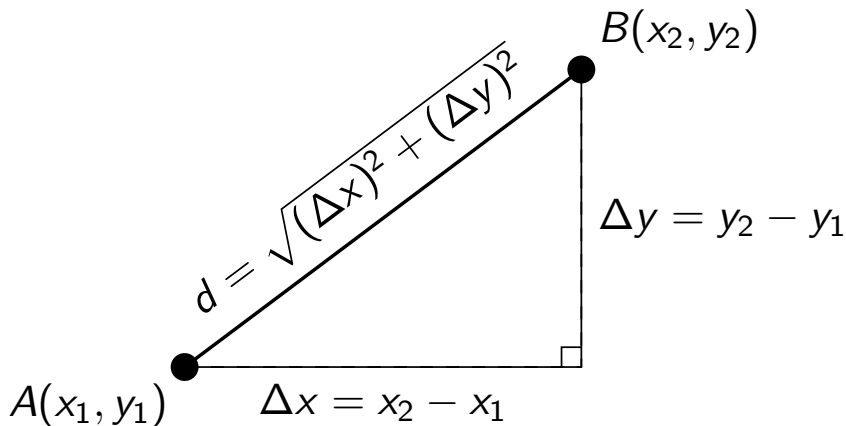
print("Meses transcurridos:", months)
```

Ejemplo: distancia entre dos puntos I

Enunciado

Manuel necesita un pequeño programa que calcule la distancia entre dos puntos (x_1, y_1) y (x_2, y_2) del plano cartesiano.

Ejemplo: distancia entre dos puntos II

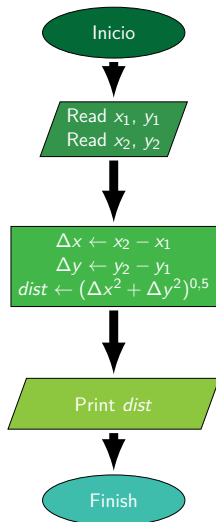


Ejemplo: distancia entre dos puntos III

Planeación: diagrama de flujo

Antes de programar, planeamos el algoritmo con un diagrama de flujo:

Ejemplo: distancia entre dos puntos IV



Ejemplo: distancia entre dos puntos V

```
# Distancia entre dos puntos
x1 = float(input("x1: "))
y1 = float(input("y1: "))
x2 = float(input("x2: "))
y2 = float(input("y2: "))

delta_x = x2 - x1
delta_y = y2 - y1
dist = (delta_x**2 + delta_y**2)**0.5

print("Distancia:", dist)
```

¡Ahora inténtalo tú!

Ejercicio 1: área del triángulo

Retoma el diagrama del área del triángulo (sección *Algoritmos: base y altura como entrada*). Escribe el programa en Python que pida base y altura por teclado y muestre el área.

Ejercicio 2: de Celsius a Fahrenheit

Diseña el algoritmo completo (pseudocódigo + diagrama de flujo) y luego el código en Python de un programa que convierta una temperatura de grados Celsius a Fahrenheit:

$$F = C \times \frac{9}{5} + 32$$

- **Entrada:** temperatura en grados Celsius.
- **Salida:** temperatura equivalente en grados Fahrenheit.

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
 - Principio de Ejecución Secuencial
 - Estado de un Programa
 - Ejecución Concurrente
- 5 Strings
- 6 Condicionales
- 7 Prueba de Escritorio

Contenido II

8 Bibliografía

Ejecución Secuencial

Definición

Un programa se ejecuta **instrucción por instrucción**, en el mismo orden en que fue escrito, de arriba hacia abajo. Es el mismo principio que ya usábamos en el pseudocódigo y en los diagramas de flujo.

```
print("Paso 1")
x = 5
print("Paso 2, x vale", x)
y = x + 1
print("Paso 3, y vale", y)
```

El orden importa

Una variable debe existir (haber sido asignada) **antes** de usarse. Si intercambiamos las líneas 2 y 3 del ejemplo, x todavía no existiría al ejecutar print y Python lanzaría un error (NameError).

Estado de un Programa

Definición

El **estado** de un programa, en un instante dado, es el conjunto de valores que tienen todas sus variables en ese momento. **Recuerden:** el operador de asignación = es el único que cambia el estado.

```
x = 5
y = x + 1
x = y * 2
```

¿Cómo cambia el estado, línea a línea?

Línea	Instrucción	x	y
1	x = 5	5	–
2	y = x + 1	5	6
3	x = y * 2	12	6

Esta tabla es, en pequeño, la idea central de la **prueba de escritorio** que veremos más

Ejecución Concurrente: una mirada rápida

Secuencial

Una instrucción **espera** a que la anterior termine para poder ejecutarse. Es lo que hemos hecho hasta ahora.

Concurrente

Varias tareas **avanzan a la vez** (o intercaladas), sin esperarse entre sí.

Ejemplo cotidiano

Mientras un navegador descarga un archivo, la interfaz sigue respondiendo a los clics: son dos tareas avanzando “al mismo tiempo”.

En este curso

Trabajaremos siempre con programas **secuenciales**. La ejecución concurrente se estudia con más profundidad en cursos posteriores; por ahora basta con reconocer que existe y en qué se diferencia.

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings**
 - Strings en Python
 - Indexación y Segmentación de Strings
- 6 Condicionales
- 7 Prueba de Escritorio
- 8 Bibliografía

Strings: cadenas de texto

Recordemos

`str` es el tipo de dato para **texto**. Se escribe entre comillas simples `'...'` o dobles `"..."`.

```
nombre = "Ana"  
apellido = 'Gomez'
```

Concatenar (+) y repetir (*)

```
saludo = "Hola, " + nombre + "!"  
print(saludo)           # Hola, Ana!  
print("Ja" * 3)         # JaJaJa  
print(len("Python"))    # 6 (longitud del string)
```

Strings y números: cuidado con +

¡Importante!

No se puede **sumar** un str con un int/float directamente: es un error de tipos (TypeError).

```
edad = 20
# print("Edad: " + edad)           # Error! (TypeError)
print("Edad: " + str(edad))        # OK: convertimos edad
    a str
print("Edad:", edad)               # OK: print acepta
    varios tipos
```

Con `print(a, b, ...)` Python convierte todo a texto automáticamente; con `+` entre strings, la conversión hay que hacerla nosotros con `str()`.

Indexación de Strings

Cada carácter tiene una posición

El primer carácter está en el índice 0; los índices **negativos** cuentan desde el final (-1 es el último).

P	y	t	h	o	n
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

```
palabra = "Python"
palabra[0]      # 'P'
palabra[5]      # 'n'
palabra[-1]     # 'n'   (ultimo character)
```

Segmentación de Strings (*slicing*)

Sintaxis

`cadena[inicio:fin]` extrae una porción, **sin incluir** el índice `fin`.
También existe `cadena[inicio:fin:paso]`.

```
palabra = "Python"
palabra[0:3]      # 'Pyt'
palabra[:3]       # 'Pyt'      (desde el inicio)
palabra[3:]       # 'hon'      (hasta el final)
palabra[:]        # 'Python'   (copia completa)
palabra[::-1]     # 'nohtyP'   (invertido, paso -1)
```

¡Ahora inténtalo tú!

Ejercicio: Strings

Dada la variable:

```
frase = "Informatica es divertida"
```

Escribe el código Python para:

- 1 Obtener el primer carácter.
- 2 Obtener los últimos 9 caracteres ("divertida") usando índices negativos.
- 3 Obtener la frase invertida.
- 4 Calcular cuántos caracteres tiene la frase.

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales**
 - Condicionales y Variables Lógicas (if, else)
 - Condicionales Anidados (elif, if-then-if)
 - Operadores Lógicos
 - Operador `in`
- 7 Prueba de Escritorio

Contenido II

8 Bibliografía

Variables Lógicas (bool)

Recordemos

Los **operadores relacionales** (`>`, `<`, `==`, ...) que ya vimos devuelven un valor de tipo `bool`: `True` o `False`.

```
mayor_de_edad = 18 >= 18
print(mayor_de_edad)           # True
print(type(mayor_de_edad))     # <class 'bool'>
```

Una variable booleana también puede asignarse directamente: `activo = True`.

El condicional if

Sintaxis

if *condición*: seguido de un bloque **indentado** que se ejecuta solo si la condición es True.

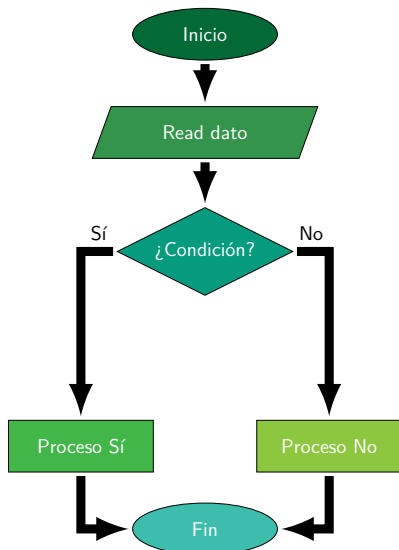
```
edad = 20
if edad >= 18:
    print("Es mayor de edad")
print("Este mensaje siempre se imprime")
```

¡Importante! La indentación no es opcional

En Python, la **sangría** (normalmente 4 espacios) es la que define qué instrucciones pertenecen al bloque del if. Un error de indentación es uno de los más comunes al empezar.

if - else

```
edad = 15
if edad >= 18:
    print("Es mayor de
          edad")
else:
    print("Es menor de
          edad")
```



El rombo del diagrama es exactamente el símbolo de **decisión** que vimos al inicio de la unidad.

elif: varias condiciones

Sintaxis

elif (“else if”) permite encadenar varias condiciones. Python evalúa en orden y ejecuta el bloque de la **primera** que sea True; el resto se ignora.

```
nota = 3.5
if nota >= 4.5:
    print("Excelente")
elif nota >= 3.5:
    print("Aprobado")
else:
    print("Reprobado")
```

if anidado (*if-then-if*)

Idea

Un if puede contener **otro** if dentro de su bloque, para preguntar algo solo cuando la primera condición ya se cumplió.

```
edad = 20
tiene_permiso = True

if edad >= 18:
    if tiene_permiso:
        print("Puede ingresar")
    else:
        print("Falta el permiso")
else:
    print("No cumple la edad minima")
```

¿Se podría escribir este mismo ejemplo con elif? No siempre: elif encadena alternativas de **una misma** pregunta; anidar sirve cuando la segunda pregunta solo tiene sentido **dentro** de la primera.

Operadores Lógicos: and, or, not

Operador	Significado	Ejemplo	Resultado
and	Y (ambas verdaderas)	True and False	False
or	O (al menos una verdadera)	True or False	True
not	Negación	not True	False

Combinándolos con if

```
edad = 20
tiene_permiso = True

if edad >= 18 and tiene_permiso:
    print("Puede ingresar")
```

Con and/or muchas veces podemos reemplazar un if anidado por un único if con la condición combinada.

Operador in

¿Para qué sirve?

Verifica si un valor está **contenido** dentro de una secuencia (por ahora, un str); devuelve True o False.

```
palabra = "murcielago"
print("a" in palabra)      # True
print("z" in palabra)      # False
print("ciela" in palabra)  # True (substring)
```

Uso típico en un condicional

```
correo = input("Escribe tu correo: ")
if "@" in correo:
    print("Parece un correo valido")
else:
    print("Falta el simbolo @")
```

Validaciones con and / or

Validar con and: varias condiciones a la vez

Un correo “válido” (de forma simple) necesita **el símbolo @ y un punto**. Con and exigimos que **ambas** se cumplan.

```
correo = input("Escribe tu correo: ")
if "@" in correo and "." in correo:
    print("Parece un correo valido")
else:
    print("Correo invalido")
```

Validar con or: cualquiera de varias opciones

Con or basta con que se cumpla **al menos una** condición.

```
dia = input("Que dia es hoy? ")
if dia == "sabado" or dia == "domingo":
    print("Es fin de semana")
else:
    print("Es dia habil")
```

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales
- 7 Prueba de Escritorio**
- 8 Bibliografía

Definición

La prueba de escritorio es una técnica manual para simular la ejecución de un algoritmo paso a paso, con el objetivo de verificar su correcto funcionamiento y detectar errores lógicos **antes** de programarlo.

Pasos

- 1 Escribir el algoritmo (o el diagrama de flujo) de forma clara.
- 2 Asignar valores iniciales a las variables de entrada.
- 3 Simular cada paso y registrar cómo cambia el **estado** del programa.
- 4 Comparar el resultado final con lo que se espera.

Ejemplo: ¿el número es par? I

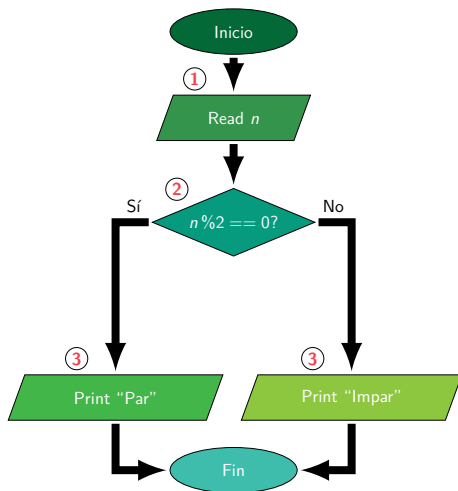
Enunciado

Diseñar un algoritmo que, dado un número entero n , indique si es **par** o **impar**.

Planeación: diagrama de flujo

Usamos el símbolo de **decisión** para preguntar si el residuo de n entre 2 es cero. Los círculos **1**, **2** y **3** marcan los pasos que vamos a seguir en la prueba de escritorio.

Ejemplo: ¿el número es par? II



Ejemplo: ¿el número es par? III

Prueba de escritorio, con $n = 7$

Seguimos los pasos **1**, **2** y **3** marcados en el diagrama de la página anterior, registrando cada variable en su propia columna:

Paso	Instrucción	Condición	n	Salida
1	$n = 7$	—	7	—
2	$n \% 2 == 0$	$7 \% 2 = 1 \rightarrow \text{False}$	7	—
3	rama else	—	7	"Impar"

Predicción

Antes de programar, ya sabemos qué **debería** imprimir el programa para $n = 7$: "Impar".

Ejemplo: ¿el número es par? IV

Del diagrama al código

```
n = int(input("Ingresa un numero: "))
if n % 2 == 0:
    print("Par")
else:
    print("Impar")
```


Ejemplo: ¿el número es par? V

Depurando con Python

Ejecuta el programa con $n = 6$ y **compara** la salida real con la que predijiste en la prueba de escritorio.

- ¿Coinciden? La lógica del programa es correcta (para este caso).
- ¿No coinciden? Revisa: ¿la condición está bien escrita? ¿la indentación agrupa las líneas correctas? ¿usaste `==` y no `=`?

Comparar predicción escrita contra resultado real es la base de **depurar** (*debug*) un programa.

¡Ahora inténtalo tú! Ejercicio integrador

Año bisiesto

Un año es bisiesto si es divisible por 4; excepto si es divisible por 100, en cuyo caso también debe ser divisible por 400 para seguir siendo bisiesto.

Pasos a entregar

- ➊ **Pseudocódigo:** lista, en tus palabras, los pasos desde el ingreso del año hasta el mensaje de salida.
- ➋ **Diagrama de flujo:** usa los símbolos vistos (incluida la decisión) para representar el algoritmo completo.
- ➌ **Prueba de escritorio:** simúlalo a mano para dos años, uno bisiesto y uno que no (por ejemplo, 2024 y 2023).
- ➍ **Código en Python:** usando `if/elif/else` y operadores lógicos (`and`, `or`), escribe el programa y verifica que su salida coincida con tu prueba de escritorio.

Contenido I

- 1 Solución de Problemas
- 2 Algoritmos
- 3 Algoritmos en Python
- 4 Ejecución de Programas
- 5 Strings
- 6 Condicionales
- 7 Prueba de Escritorio
- 8 Bibliografía**

Para profundizar

Si quieres reforzar lo visto en esta unidad, estos recursos son un buen punto de partida:

- **Libro:** [1], un libro de introducción a Python muy usado, con proyectos prácticos paso a paso.
- **Página web:** [3], el tutorial oficial y gratuito de Python.



Eric Matthes.

Python Crash Course: A Hands-On, Project-Based Introduction to Programming.
No Starch Press, 3 edition, 2023.



George Polya.

How to solve it: A new aspect of mathematical method.
In *How to solve it*. Princeton university press, 2014.



Python Software Foundation.

The python tutorial.
<https://docs.python.org/3/tutorial/index.html>, 2024.
Documentación oficial de Python.